

TD7: Ingeniería de Datos

Clase Práctica 8: Calidad de datos

Clase de hoy

- Repaso gobernanza / calidad de datos
- Great Expectations

(Break)

- Optimización de consultas
- Consultas + entrega del TP1

Repaso - Verdadero o Falso

Pregunta 1

La consistencia se verifica cuando se comprueba que el formato de un dato es el correcto, como en el caso de la fecha, mientras que la validez se enfoca en que los datos no violen reglas de negocio internas, como la coherencia entre direcciones registradas.

Pregunta 1

La consistencia se verifica cuando se comprueba que el formato de un dato es el correcto, como en el caso de la fecha, mientras que la validez se enfoca en que los datos no violen reglas de negocio internas, como la coherencia entre direcciones registradas.

Falso - Es al revés: la validez se enfoca en que los valores estén en los rangos y tipos correctos, y la consistencia se refiere a que los datos no violen reglas de negocio y sean coherentes entre sí.

Pregunta 2

La gobernanza de datos abarca la definición de responsables (*data steward*) y las normas de uso, pero la responsabilidad de asegurar *High Availability* (Alta Disponibilidad) recae exclusivamente en la fase de almacenamiento del ciclo de vida del dato.

Pregunta 2

La gobernanza de datos abarca la definición de responsables (*data steward*) y las normas de uso, pero la responsabilidad de asegurar *High Availability* (Alta Disponibilidad) recae exclusivamente en la fase de almacenamiento del ciclo de vida del dato.

Verdadero - La gobernanza abarca las definiciones y normas. La High Availability y la Availability son consideraciones clave mencionadas específicamente para la fase de Almacenamiento.

Pregunta 3

Según el ciclo de vida del dato, la fase de compartir es la más difícil de controlar porque es donde se debe auditar quién leyó y modificó el dato para aplicar el Principio de Privilegio Mínimo.

Pregunta 3

Según el ciclo de vida del dato, la fase de compartir es la más difícil de controlar porque es donde se debe auditar quién leyó y modificó el dato para aplicar el Principio de Privilegio Mínimo.

Falso - La fase más difícil de controlar es compartir por los canales no oficiales; el Principio de Privilegio Mínimo y la auditoría se aplican principalmente en la fase de Uso.

Pregunta 4

El *Data Lineage* (Linaje de datos) es una técnica clave dentro de la observabilidad, ya que permite trazar la historia de un dato, lo que ayuda a realizar el Análisis de Causa Raíz (RCA) de las anomalías.

Pregunta 4

El *Data Lineage* (Linaje de datos) es una técnica clave dentro de la observabilidad, ya que permite trazar la historia de un dato, lo que ayuda a realizar el Análisis de Causa Raíz (RCA) de las anomalías.

Verdadero - La observabilidad utiliza *Data Lineage* y Análisis de Causa Raíz para prevenir, detectar y resolver anomalías.

Pregunta 5

La precisión de un dato se verifica al revisar si sus valores están dentro de los rangos correctos (ej.: un monto no es 'a'), mientras que la conformidad asegura que el dato se corresponda con la realidad observable (ej.: una dirección es correcta).

Pregunta 5

La precisión de un dato se verifica al revisar si sus valores están dentro de los rangos correctos (ej.: un monto no es 'a'), mientras que la conformidad asegura que el dato se corresponda con la realidad observable (ej.: una dirección es correcta).

Falso - La validez verifica los rangos correctos, la precisión verifica la correspondencia con la realidad, y la conformidad asegura que esté en el formato correcto.

Great Expectations

Setup

- Ir al campus -> practica_data_quality.zip
- Seguir la guía “8.1 Calidad de datos”

Optimización de consultas

Ejercicio 0 - Warm Up

```
SELECT *  
FROM Sales.Customer  
WHERE CustomerID = 11000;
```

```
SELECT *  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

```
SELECT CustomerID  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

Ejercicio 0 - Warm Up

```
SELECT *  
FROM Sales.Customer  
WHERE CustomerID = 11000;
```

```
SELECT *  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

```
SELECT CustomerID  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

- 1) **Clustered Index** (PK_Customer_CustomerID)
Se buscan todos los atributos por la PK
(Clustered Index Seek)

Ejercicio 0 - Warm Up

```
SELECT *  
FROM Sales.Customer  
WHERE CustomerID = 11000;
```

```
SELECT *  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

```
SELECT CustomerID  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

1) **Clustered Index** (PK_Customer_CustomerID)
Se buscan todos los atributos por la PK
(Clustered Index Seek)

2) **Unclustered Index**
AccountNumber tiene un Unclustered Index
(AK_Customer_AccountNumber), entonces se
lo usa para un Index Seek y buscar la
clave (CustomerID) con un Key Lookup y
así traer la fila.

Ejercicio 0 - Warm Up

```
SELECT *  
FROM Sales.Customer  
WHERE CustomerID = 11000;
```

```
SELECT *  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

```
SELECT CustomerID  
FROM Sales.Customer  
WHERE AccountNumber = 'AW00011000';
```

- 1) **Clustered Index** (PK_Customer_CustomerID)
Se buscan todos los atributos por la PK (Clustered Index Seek)
- 2) **Unclustered Index**
AccountNumber tiene un Unclustered Index (AK_Customer_AccountNumber), entonces se lo usa para un Index Seek y buscar la clave (CustomerID) con un Key Lookup y así traer la fila.
- 3) **Unclustered Index**
Como en este caso se piden atributos que forman parte del Unclustered Index, se resuelve directamente con un Index Seek sobre el mismo.

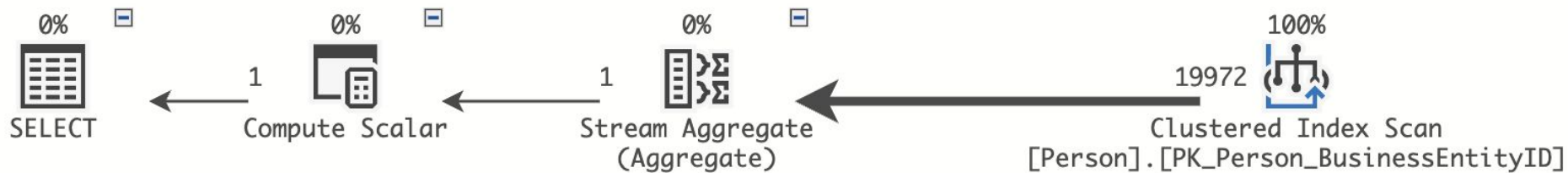
Ejercicio 1

```
select count(Title) from Person.Person;
```

```
select count(FirstName) from Person.Person;
```

Ejercicio 1

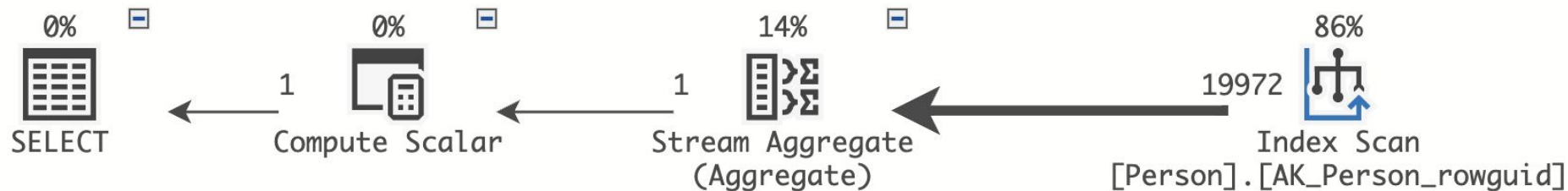
```
select count(Title) from Person.Person;
```



Como Title es nulleable, debe realizar un Clustered Index Scan usando la PK. Tiene que fijarse por cada fila si es NULL o no para contar.

Ejercicio 1

```
select count(FirstName) from Person.Person;
```



Ahora bien, `FirstName` no es nulleable, por lo que contar cuántos registros no son nulos = contar todas las filas. Entonces, usá un Index Scan con el índice Unclustered `AK_Person_rowguid` al ser más pequeño.

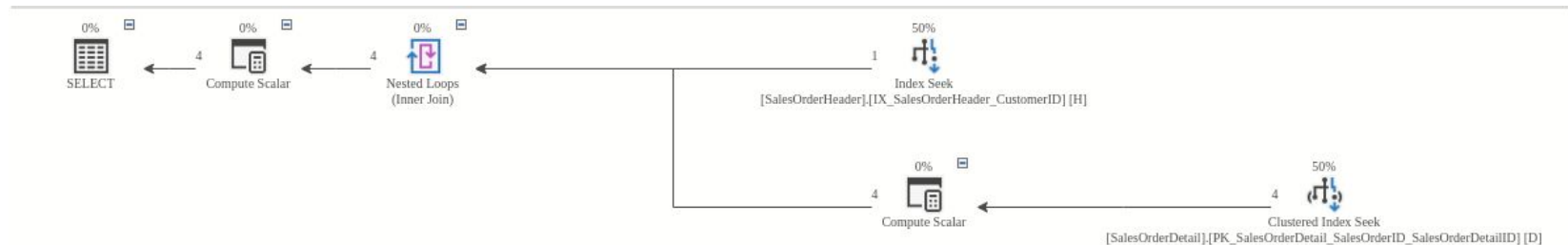
Ejercicio 2

```
select H.CustomerID, H.SalesOrderID, D.ProductID, D.LineTotal
from Sales.SalesOrderHeader H
inner join Sales.SalesOrderDetail D on H.SalesOrderID = D.SalesOrderID
where H.CustomerID = 670
```

```
select H.CustomerID, H.SalesOrderID, D.ProductID, D.LineTotal
from Sales.SalesOrderHeader H
inner join Sales.SalesOrderDetail D on H.SalesOrderID = D.SalesOrderID
where H.CustomerID <> 670
```


Ejercicio 2

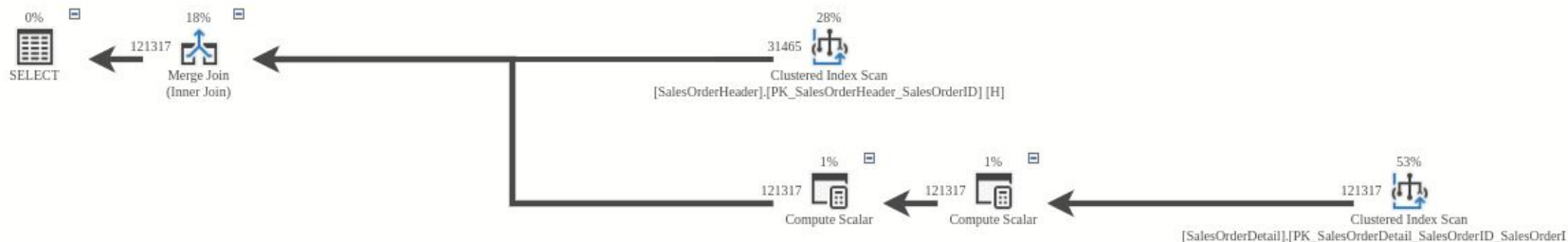
```
select H.CustomerID, H.SalesOrderID, D.ProductID, D.LineTotal
from Sales.SalesOrderHeader H
inner join Sales.SalesOrderDetail D on H.SalesOrderID = D.SalesOrderID
where H.CustomerID = 670
```



- 1 - Se hace un seek del customer id para SalesOrderHeader al tener un índice unclustered.
- 2 - Con el resultado de ese seek, se trae del índice unclustered de SalesOrderHeader la clave clustered (su PK es SalesOrderId), y con esa se hace un clustered index seek sobre SalesOrderDetail
- 3 - Último paso, inner join

Ejercicio 2

```
select H.CustomerID, H.SalesOrderID, D.ProductID, D.LineTotal
from Sales.SalesOrderHeader H
inner join Sales.SalesOrderDetail D on H.SalesOrderID = D.SalesOrderID
where H.CustomerID <> 670
```



- 1 - Al no tratarse de una comparación por igualdad, se realiza un clustered index scan en cada lado buscando las filas con CustomerId <> 678. No se puede hacer un seek por desigualdad.
- 2 - Los resultados vienen ordenados, con lo cual se puede joinear utilizando el Merge Join más eficiente en comparación

Ejercicio 3

```
select e.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
    on e.BusinessEntityID = h.BusinessEntityID
group by e.BusinessEntityID
```

```
select h.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
    on e.BusinessEntityID = h.BusinessEntityID
group by h.BusinessEntityID
```

Ejercicio 3

```
select e.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
        on e.BusinessEntityID = h.BusinessEntityID
group by e.BusinessEntityID
```

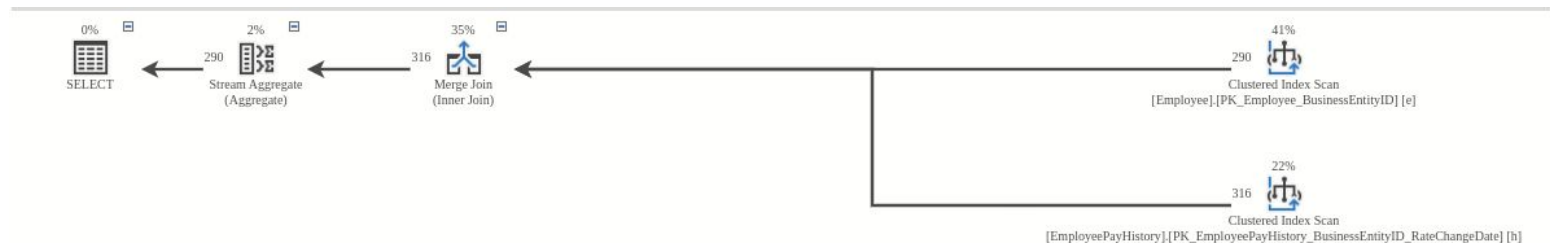
```
select h.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
        on e.BusinessEntityID = h.BusinessEntityID
group by h.BusinessEntityID
```

Diferencias:

- Group by en Employee e vs en EmployeePayHistory

Ejercicio 3

```
select e.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
    on e.BusinessEntityID = h.BusinessEntityID
group by e.BusinessEntityID
```



- 1 - Utiliza la PK Employee BusinessEntityID haciendo un Clustered Index Scan
- 2 - Revisa Clustered Index Scan en EmployeePayHistory con la clave (BusinessEntityId, RateChangeDate)
- 3 - Merge Join ya que ambas vienen ordenadas (ambas por BusinessEntityId) y condición de igualdad

Ejercicio 3

```
select h.BusinessEntityID, sum(rate) from HumanResources.Employee e
inner join HumanResources.EmployeePayHistory h
    on e.BusinessEntityID = h.BusinessEntityID
group by h.BusinessEntityID
```



- 1 - Al hacer group by por BusinessEntityId, en la tabla EmployeePayHistory tenemos a BusinessEntityId como parte de la PK. Su PK ya es (BusinessEntityId, RateChangeDate)
- 2 - El planner sabe que no hace falta ir a la tabla Employee. Todos sus BusinessEntityId ya están en EmployeePayHistory. La 2da parte de la key es un atributo.
- 3 - Escanea sólo esa tabla para hacer la agregación.